

Project Requirements

Library Management System | Python + Raw SQL (PostgreSQL)

1. Purpose of This Document

This document lists what your Library Management System project should include. Unlike the Expense Tracker, this project must use multiple related tables and requires you to practice joins, foreign keys, and basic relational database design. No code is provided here — the goal is for you to design and write the program yourself using Python and raw SQL queries via `psycopg2`, connecting to a PostgreSQL database.

2. What the Program Should Do

Build a command-line program that manages a small library: its books, authors, borrowers (members), and the record of who currently has which book borrowed. The user interacts through a text menu, and all data is stored in and retrieved from a PostgreSQL database using raw SQL queries — not an ORM.

Core features the program must support:

- Add a new author to the system
- Add a new book, linked to an existing author
- Add a new borrower (library member)
- Record a book being borrowed by a member (a loan)
- Record a book being returned (mark the loan as completed / remove it as active)
- View a list of all books, showing each book's author
- View a list of all books that are currently borrowed, showing who has them
- View a list of all books currently borrowed by a specific member
- Search for a book by title or by author name
- Prevent a book from being borrowed if it is already checked out by someone else

3. Database Design Requirements

This project is primarily about relational thinking, not just CRUD. Your schema must include at least four related tables, structured with proper keys:

Table	Purpose	Key Relationships
authors	Stores author details (name, etc.)	Primary key referenced by books
books	Stores book details (title, author, etc.)	Foreign key to authors
borrowers	Stores library member details (name, contact info)	Primary key referenced by loans
loans	Tracks which book is borrowed by which member and when	Foreign keys to both books and borrowers

Design rules to follow:

- Every table must have a proper primary key (an auto-incrementing id is fine).
- Relationships between tables must be enforced using foreign keys, not just matching text values.
- A book should not store the author's name directly — it should store a reference (author_id) to the authors table.
- The loans table should record which book, which borrower, the date borrowed, and the date returned (which can be empty/null if the book has not yet been returned).
- Decide and be able to explain what should happen to loan records if a book or borrower is deleted (this is a real design decision, not a technical detail to skip over).

4. SQL Skills This Project Must Demonstrate

- CREATE TABLE statements with primary keys, foreign keys, and appropriate data types
- INSERT, UPDATE, and DELETE statements for managing books, authors, borrowers, and loans
- SELECT queries using INNER JOIN to combine data across at least two tables (for example, showing each book with its author's name)
- A query using LEFT JOIN, where relevant (for example, listing all books, including ones with no current loan)
- Filtering with WHERE, and searching using pattern matching (for example, ILIKE for case-insensitive search)
- At least one query using an aggregate function with GROUP BY (for example, counting how many books each borrower currently has)
- Parameterized queries throughout — user input must never be inserted directly into a SQL string

5. Python Program Structure Requirements

- The database connection logic should be separated from the menu/interaction logic (for example, a dedicated function or module for connecting to the database).
- Each feature (adding a book, recording a loan, searching, etc.) should be its own function, not one large block of code.
- Input should be validated before being sent to the database (for example, checking that an author exists before adding a book for them).
- Errors from the database (such as a failed connection or a constraint violation) should be handled gracefully, not crash the program.
- Database credentials should not be hard-coded directly in the script; use environment variables or a separate config file that is not committed to version control.

6. Deliverables Checklist

- A SQL script (or set of CREATE TABLE statements) documenting the full schema, including keys and relationships
 - The main Python program implementing all core features listed above
 - A short README explaining the schema (what each table is for and how they relate), and how to run the program
 - A brief walkthrough (can be verbal, with you) demonstrating each core feature working, including the case where a book cannot be borrowed because it is already checked out
-

Tip: Design the schema on paper first (tables, columns, keys, relationships) before writing any SQL or Python. Getting the relationships right up front will make every feature after that much easier to implement.